

FORMATION EMBARQUÉE

Antisèche — Git Terminal

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — Git & terminal

Git : les 12 commandes qui suffisent

```
git init                # créer un dépôt dans le dossier courant
git clone <url>          # récupérer un dépôt existant
git status              # QUE se passe-t-il ? (à taper tout le temps)
git add fichier.c / git add .      # préparer (stager)
git commit -m "message clair"      # enregistrer
git log --oneline -10             # historique court
git diff / git diff --staged      # ce qui a changé
git push / git pull              # envoyer / récupérer
git branch ma-branche && git switch ma-branche # (ou checkout -b)
git restore fichier.c            # annuler mes modifs NON committées
git revert <sha>                 # annuler un commit déjà partagé (propre)
```

Messages de commit utiles

```
TP1 seance 2 : OLED + decoupage en modules
fix: reamorcner Receive_IT (on ne recevait qu'un octet)
TD 04 ex.3 : anti-rebond + synchroniseur, testbench vert
```

Impératif présent, court, ce qui change **et pourquoi**. Un commit par séance de travail minimum.

Se sortir des ennuis

```
git commit --amend -m "message corrigé"    # corriger le DERNIER commit (non poussé)
git reset --soft HEAD~1                    # défaire le dernier commit, garder les fichiers
git stash / git stash pop                  # mettre de côté / reprendre
git checkout -- .                          # tout jeter (Δ irréversible)
```

.gitignore type pour l'embarqué

```
*.o
*.elf
*.hex
*.bin
*.map
Debug/
build/
__pycache__/
*.class
*.cf          # GHDL
*.ghw         # ondes GTKWave
.vscode/
```

Terminal Linux/macOS — le minimum vital

```
pwd                # où suis-je ?
ls -la             # lister (avec cachés + détails)
cd dossier / cd .. / cd ~
mkdir -p a/b/c     # créer une arborescence
cp src dst / mv src dst / rm fichier / rm -r dossier
cat f.txt / less f.txt / head -20 f / tail -f journal.log
grep -rn "motif" .  # chercher dans les fichiers
find . -name "*.c"  # chercher des fichiers
chmod +x script.sh  # rendre exécutable
```

Série & matériel

```
ls /dev/tty*       # trouver le port (ttyUSB0, ttyACM0)
dmesg | tail        # "quel port ma carte vient-elle de prendre ?"
sudo usermod -aG dialout $USER # droits port série (Linux, puis se reconnecter)

picocom -b 115200 /dev/ttyACM0 # terminal série (quitter : Ctrl-A Ctrl-X)
screen /dev/ttyACM0 115200     # (quitter : Ctrl-A puis K)
minicom -D /dev/ttyUSB0 -b 115200
```

Sous **Windows** : PuTTY ou le moniteur série de l'IDE (port **COMx**).

Compilation & outils

```
make                # construire      make clean / make test
gcc -Wall -Wextra -O2 f.c -o app
hexdump -C firmware.bin | head    # inspecter un binaire
arm-none-eabi-objdump -d f.elf     # désassembler
arm-none-eabi-size f.elf           # taille flash/RAM utilisée
```

Python (bancs de test, Modbus, tracés)

```
python3 -m venv .venv && source .venv/bin/activate # env isolé
pip install pyserial pymodbus matplotlib
python3 script.py
```

```
import serial                # parler à la carte
s = serial.Serial('/dev/ttyACM0', 115200, timeout=1)
s.write(b'status\n'); print(s.readline().decode())
```

Raccourcis terminal qui font gagner du temps

Touches	Effet
<code>Tab</code>	complète le nom (le réflexe n°1)
<code>↑</code> / <code>Ctrl-R</code>	commande précédente / recherche dans l'historique
<code>Ctrl-C</code>	interrompre le programme en cours
<code>Ctrl-A</code> / <code>Ctrl-E</code>	début / fin de ligne
<code>Ctrl-L</code>	effacer l'écran
<code>!!</code>	rejouer la dernière commande (<code>sudo !!</code>)